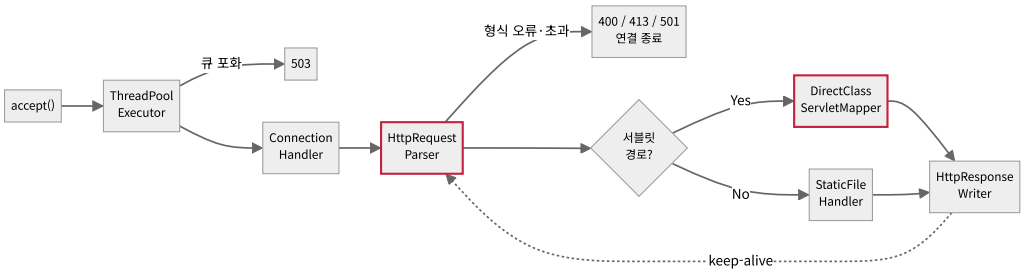
HTTP/1.1 스펙 기반, Java 소켓으로 직접 구현한 WAS (Web Application Server)

2026.06 ~ 2026.07 개인 프로젝트 Java 17, Maven, JUnit 4, Logback github.com/sun8183/was-lab

HTTP/1.1(GET/HEAD/POST, keep-alive), 가상호스트, 정적 파일 스트리밍과 ETag, 스레드풀 포화 시 503, graceful shutdown을 직접 구현하며 WAS의 내부 구조와 동작 방식을 학습



1. HTTP 커넥션·스레드 관리

배경

- 요청마다 스레드를 만들면 비용이 크고 무제한으로 늘어나며, keep-alive로 스레드 점유 시간도 길어지는 문제 발생

구현

사전 생성	스레드 50개를 서버 시작 시 미리 생성(`prestartAllCoreThreads`)해 초기값 설정
확장 순서	기본 스레드 소진 → 큐(100) 대기 → 큐도 차면 max 200까지 증설. 증설분은 60초 유휴 시 회수
포화 거절	`AbortPolicy` 예외를 accept 스레드가 받아 503 즉시 응답. 무제한 대기와 메모리 증가 차단
크기 설정	`config.json` 으로 설정 값 분리. 최악의 경우 바디 메모리(max 200 × 1MB = 200MB)가 힙에 들어오는지로 판단하여 설정
Keep-Alive	한 연결에서 여러 요청을 처리하고, 소켓 `SO_TIMEOUT` 으로 20초간 다음 요청이 없으면 연결 회수
종료 처리	graceful shutdown. 종료 신호 후 신규 연결은 503, 처리 중인 요청은 최대 30초 기다리고 초과 시 강제 종료한 뒤 `destroy()` 호출

결과

- Keep-Alive 연결 재사용으로 연결 수립 비용을 줄이고, 스레드 운영 정책(풀 크기 제한, 초과 요청 503 거절, 유휴 연결 회수)으로 과부하에서도 멈추지 않고 처리

2. HTTP 요청 파싱과 메서드 처리

배경

- 소켓으로 들어오는 바이트를 HTTP/1.1 규칙에 따라 요청 라인·헤더·바디로 직접 나눠야 함

구현

요청 라인	첫 줄을 공백으로 메서드·경로·버전 3개로 분리. 형식 오류는 400, 지원하지 않는 메서드는 501
헤더	빈 줄까지 `이름: 값` 줄을 읽어 대소문자 무시 맵에 저장. `ISO-8859-1` 로 손실 없이 디코딩
바디	`Content-Length` 만큼만 바이트로 읽음. 상한 초과는 읽기 전에 413, chunked는 501
GET / HEAD	정적 파일과 서블릿을 조회. HEAD는 GET과 같은 헤더에서 바디만 빼고 응답
POST	서블릿으로만 처리. 폼은 `getParameter()` , JSON 등은 `getBody()` 원본 바이트로 전달

결과

- GET·HEAD·POST를 처리하고, 멀티바이트·바이너리 바디도 손실 없이 수신. 잘못된 요청은 400·413·501로 구분 응답

3. 서블릿 컨테이너

배경

- 프레임워크 없이 URL과 서블릿을 연결하고, 생명주기와 동시 접근을 직접 관리해야 함

구현

API	`SimpleServlet` 인터페이스(`init` / `service` / `destroy`). 요청은 `getParameter()` · `getHeader()` · `getBody()` , 응답은 `getWriter()` 로 작성
매핑	URL 경로(`/Hello`)를 클래스명으로 리플렉션 로딩. `ServletMapper` 인터페이스로 방식 교체 가능
안전한 로딩	`initialize=false` 로 로딩만 하고 서블릿 타입만 초기화 → 임의 클래스의 static 코드 실행 차단
생명주기	첫 요청 때 `init()` 하는 지연 초기화. `computeIfAbsent` 로 클래스당 인스턴스 하나 보장
실패 캐시	서블릿이 아닌 경로를 1,000개 상한 캐시에 기록해 반복 클래스 탐색 제거

결과

- 클래스만 추가하면 설정 없이 URL로 호출. 로딩 실패는 404(서블릿 없음)와 500(생성 실패)으로 구분

4. 가상호스트와 정적 파일 서빙

배경

- 서버 하나로 Host별 사이트를 분리하고, 큰 파일도 힙 부담 없이 전송해야 함

구현

가상호스트	Host 헤더에서 포트를 떼고 소문자로 바꿔 `Map` 조회(O(1)). 미매칭은 설정 순서상 첫 호스트로 폴백(`LinkedHashMap`)
스트리밍	`Content-Length` 는 `Files.size()` 로 먼저 구하고, 바디는 `transferTo()` 로 8KB 단위 전송
ETag	수정 시각 + 크기로 생성, `If-None-Match` 일치 시 304. 내용 해시는 매 요청 파일 전체를 읽어야 해 제외
에러 페이지	vhost별 403/404/500 HTML 지정. 파일이 없거나 루트를 벗어나면 기본 텍스트로 응답
전송	헤더·바디를 `BufferedOutputStream` 에 모아 `flush()` 시 한 번에 전송

결과

- 설정만으로 호스트별 사이트·에러 페이지 분리. 힙 사용량이 파일 크기와 무관하게 일정하고, 변경 없는 파일은 바디 없이 304로 응답

5. 보안 규칙

배경

- URL 경로가 그대로 파일 경로가 되므로, 루트 밖 파일이나 실행 파일이 노출될 위험

구현

경로 이탈	요청 경로를 `httpRoot` 기준으로 정규화한 뒤 `startsWith(httpRoot)` 로 루트 이탈 검사
확장자 차단	파일명의 마지막 확장자를 소문자로 잘라 `Set.contains()` 로 조회. 차단 목록은 `config.json`
확장성	`ForbiddenRule` 인터페이스 하나만 구현하면 규칙 추가

결과

- `../` 경로 조작과 차단 확장자 요청을 403으로 거절. 에러 페이지 경로에도 같은 루트 검사 적용

6. 로깅

배경

- 장애 원인을 추적할 수 있어야 하고, 로그 파일이 디스크를 무한히 차지하면 안 됨

구현

롤링	logback `SizeAndTimeBasedRollingPolicy` 로 일별 폴더 분리, 파일당 10MB 분할, 30일 보관, 총 1GB 상한
레벨 분리	접근 로그를 상태 코드별로 기록. 5xx는 ERROR, 4xx는 WARN, 그 외는 INFO
예외	에러 발생 시 스택트레이스 전체 기록

결과

- ERROR 레벨만 걸러 서버 오류를 바로 확인. 로그 디스크 사용량은 1GB로 상한 보장